

Sorting: Optional for only the interested

- Advanced optimizations that pipeline operators:
- `SELECT uid, experience FROM Users ORDER BY experience`
 - At write in Pass 0 to disk (if needed):
 - Do projection on necessary attributes → each tuple is smaller → less pages
 - When you write to disk you can fill pages at 100% → even less pages
 - that is, projection and compacting pages is combined with the sort!
 - Example:
 - User has 40.000 tuples 500 data
 - Write of pass 0:
 - » $8 \text{ Bytes (uid, experience)} * 40.000 / 4000 \text{ Bytes} = 80 \text{ pages}$
 - Pass I has to read in much less

Projection Distinct Operation

```
SELECT  DISTINCT  uname  
FROM    USER
```

- Basic project often done on the fly (as discussed previously)
- More complex: **SELECT DISTINCT name FROM ...**
 - Requires sort in order to eliminate duplicates!!
 - Often done at the very end (less tuples) or whenever the relation is sorted for some other reason
 - Expensive operation: use **DISTINCT** only when really necessary

Set Operations

- Intersection and cross-product special cases of join.
- Union (Distinct) and Except similar;
- For instance: sorting based approach to union:
 - Sort both relations (on combination of all attributes).
 - Scan sorted relations and merge them.

Aggregate Operations (AVG, MIN, etc.)

- Usually done at the very last step after all selections/joins etc.
- Without grouping:
 - In general, requires scanning the relation.
- With grouping:
 - Sort on group-by attributes, then scan relation and compute aggregate for each group. (Can improve upon this by combining sorting and aggregate computation.)